Advanced Algorithms

Programming Assignment 2

Bin Packing Algorithm Comparison

Elan Blaustein – 24958098
Swas Dwivedi – 25541691

November 4, 2026

The Bin Packing Problem

In the Bin Packing (BP) problem, we're given n items with integer sizes $Y = \{y_1, y_2, ..., y_n\}$ and an unlimited supply of identical bins, each with same integer capacity of C where $0 < y_i \leq C$ for each item. The goal is optimise the packing of items to use the fewest number of bins as possible, where each item is placed in exactly one bin and the total size of the items in any bin does not exceed C. For an instance of the problem I (specified by the items y_i and capacity C), $\text{OPT}(I)$ denotes the optimal, minimum number of bins required for a valid packing.

With capacity $C = 10$ and items $\{5, 4, 4, 3, 2, 2\}$, the items may be optimally packed into two bins: $\{5, 3, 2\}$ and $\{4, 4, 2\}$, both filled exactly to capacity and therefore minimising $\text{OPT}(I) = 2$. As an optimal packing cannot exceed the capacity for any bin, we can identify a lower bound for every instance (as well as a naive upper-bound):

$$\left\lceil \frac{\sum_i y_i}{C} \right\rceil \leq \text{OPT}(I) \leq n$$

The correlated decision version of the problem asks whether a given packing instance I can be packed into (at most) k bins. Garey and Johnson (1975) proved that the decision BP problem is NP-complete, and by extension, the optimisation problem as NP-hard. They did so via reduction from the Partition problem (partitioning a set of integers into two equal sums), a known NP-complete problem. Given integers $c_1, c_2, ..., c_n$ to partition, a BP decision problem can formed by determining whether the integers may be packed into 2 bins, each with capacity $C = \frac{1}{2}\sum_i c_i$. Garey and Johnson (1979) then proved that the BP problem is Strongly NP-complete (is NP-complete even when the sizes of the parameters Y and C are bound by polynomials). This was done via a reduction from different, Strongly NP-complete problem, the 3-partition problem (partitioning a set of integers into triplets such that all have equal sums). Unlike weaker NP-complete problems (e.g. Knapsack) which can be solved in pseudo-polynomial time with techniques like Dynamic Programming, strong NP-hardness means no such algorithm exists for BP unless P = NP.

BP has various applications where a quick algorithm has practicality. One of the clearest analogues is any situation where distinct objects must be placed into large containers en masse, such as the loading of goods into freight vehicles. Another application is digital memory allocation; BP algorithms may be used to allocate files into an equal-partitioned file system to minimise unused space.

Bin Packing Algorithms

In this investigation, we compare two approaches: an **Exact** method that formulates the problem as an Integer Linear Problem (ILP), always finding OPT(I) given sufficient time, and an **Approximate** method called First Fit Decreasing, an efficient algorithm guaranteed to have an approximation ratio at most $\frac{11}{9}$ times the optimum. The algorithm implementations may be found in our GitHub repository: https://github.com/Swas26/binPacking.

Exact: Integer Linear Programming (ILP)

The Key Idea Rather than designing a search procedure by hand, we write a BP instance as a set of linear equations and inequalities, and let a general-purpose solver {HiGHS} search for the best answer. A Linear Program (LP) minimises a linear objective subject to linear constraints, where variables may take any real value. An Integer Linear Program (ILP) adds the requirement that variables take whole number values which is very essential for BP, since an item cannot be split between two bins. This condition is also what makes the problem NP-hard rather than solvable in polynomial time.

Formulation: We use the standard assignment model for Bin Packing (Delorme et al., 2016). Suppose we allow at most U bins. We use two kinds of yes/no (0 or 1) variables:

- $y_j = 1$ if bin j is used, and 0 otherwise;
- $x_{ij} = 1$ if item i is put in bin j, and 0 otherwise.

The LP model is:

$$\text{Minimise} \quad \sum_{j=1}^{U} y_j \quad \text{subject to:}$$

$$\sum_{j=1}^{U} x_{ij} = 1 \quad \text{for every item } i,$$

$$\sum_{i=1}^{n} s_i \, x_{ij} \leq C \, y_j \quad \text{for every bin } j,$$

$$x_{ij},\, y_j \in \{0, 1\}$$

The first line counts how many bins are used, which we want as small as possible. The second makes sure every item is packed exactly once i.e. for each item, its x_{ij} values across all bins must add up to 1. The third makes sure no bin is overfilled: if bin j is used ($y_j = 1$), its items can add up to at most C, and if it is not used ($y_j = 0$), nothing can be put in it. The last line says every variable must be 0 or 1, so an item cannot be split between bins; together with the second line, this means each item goes whole into exactly one bin.

Choosing U: The model needs the number of bin slots U fixed in advance. Setting $U = n$, one bin per item, always works, since every item fits in a bin on its own, but it creates n^2 assignment variables (the worst case scenario). Instead, we first run the approximate, overestimating First Fit Decreasing algorithm and use the number of bins it returns as U. Since FFD always finds a valid packing, the optimal solution literally never needs more bins than this, so $\mathrm{OPT}(I) \leq U$, which keeps the model much smaller i.e. the solver is only offered as many bin slots as it could probably need.

Symmetry breaking: Because all bins are identical, the same packing can be written in many different ways just by swapping the bin labels, for example calling bin 1 "bin 2" and bin 2 "bin 1". The solver would otherwise waste time exploring each of these copies separately. Following suggestions by Delorme et al. (2016, p. 5), we add two extra rules that remove most of these copies without excluding any genuinely distinct packing:

$$y_j \geq y_{j+1} \quad \text{for every bin } j < U,$$
$$x_{ij} = 0 \quad \text{whenever } j > i.$$

The first rule constrains the bins to be used in order, so bin 2 can only be used if bin 1 is also used. The second says item i can only go into one of the first i bins: item 1 must go in bin 1, item 2 in bin 1 or 2, and so on.

Solving: We've built the model in C++; first it generates the LP according to the above rules, and then solves the LP with HiGHS v1.15.1 (HiGHS Development Team, 2026; Huangfu & Hall, 2018), an open source solver. HiGHS solves integer programs using the branch and cut method (HiGHS Development Team, 2026).

It first solves the LP relaxation, the same model but with variables allowed to take fractional values, which gives a lower bound since the true answer can not be any better than this. It then picks a variable with a fractional value, such as $x_{ij} = 0.6$, and splits the problem into two cases (branches), one with that variable fixed to 0 and one with it fixed to 1. Each case is then solved again as an LP, and the process repeats, building a search tree.

Any branch whose bound cannot beat the best packing found so far is cut off without being explored any further and extra constraints called cutting planes are added to tighten the bounds. The search ends when the best packing is found and equals the lower bound, proving it's optimal.

We run HiGHS on a single thread, to match FFD, and with a time limit; if the limit is reached, it returns the best packing found so far, which may or may not be optimal.

Complexity: The model has $U + nU$ variables and $n + U + (U - 1)$ constraints, so its size grows only polynomially with n. Solving it, however, takes exponential time in the worst case, because the branch and cut search tree can grow exponentially with the number of variables. A known weakness of this model is that its LP relaxation gives a lower bound no stronger than,

$$\left\lceil \frac{\sum_{i=1}^{n} s_i}{C} \right\rceil$$

On instances where the optimum is above this bound, or where many near-identical packings exist, the solver may find a good packing quickly but take far longer to prove that it is in fact optimal. Delorme et al. (2016, p. 5) notes that, despite improvements such as symmetry breaking, the practical performance of this model "remains quite poor".

Approximate: First-Fit Decreasing (FFD)

There is a large variety of approximation algorithms for the BP problem, but this investigation will concern itself with the family of First-Fit (FF) algorithms (Silvano, 1990). The general concept behind FF algorithms is rather simple; bins are opened in a sequence, and the algorithm iterates through every item one by one. If an item doesn't fit in the first bin (placing it would go over capacity), the algorithm will try to place it in the next bin. This continues across all open bins, and if no bin is able to include the current item, a new bin is opened at the end of the sequence and the item is placed into it. This algorithm runs $O(n^2)$ iterations in the worst case, and is thus incredibly efficient as compared to exact, super-polynomial algorithms. Regarding approximation accuracy, the standard First-Fit algorithm on an arbitrary list of items was proven by Dósa and Sgall (2013) to produce a bin count at most 1.7 times the optimal:

$$\mathrm{FF}(I) \leq \left\lfloor \frac{17}{10}\mathrm{OPT}(I) \right\rfloor$$

To improve the accuracy bounds, an algorithm must determine the best way to minimise empty space, perhaps by placing the smallest items after all the larger items have been locked in, nestling in the capacity gaps. This intuition informs the main approximation algorithm investigated, First-Fit Decreasing. All that changes between FF and FFD is that the list of integer items is sorted beforehand, from largest to smallest. This minimally affects the algorithm complexity, $O(n \log n) \leq O(n^2)$, yet increases accuracy by a sizeable factor. Dósa (2007) proved that by pre-sorting the list, the FFD algorithm achieves an approximation factor of 1.22:

$$\mathrm{FFD}(I) \leq \frac{11}{9}\mathrm{OPT}(I) + \frac{6}{9}$$

The proof behind this statement is rather complicated, but in summary, if FFD was to surpass this upper-bound, it would be possible to create a weighted bin sum by separating the items y_i into size classes (e.g. any item larger than $\frac{C}{2}$ is in the largest size class). This weighted sum can be rearranged using optimality assumptions to arrive at the above inequality regardless, creating a contradiction.

This proof intuition was used by Johnson and Garey (1985) once more to develop a modified FFD algorithm (aptly named MFFD). This algorithm takes advantage of the item size classes by partitioning the items into groups during the preprocessed ordering phase: Tiny = $(0, \frac{C}{6}]$, Small = $(\frac{C}{6}, \frac{C}{3}]$, Medium = $(\frac{C}{3}, \frac{C}{2}]$, Large = $(\frac{C}{2}, C]$. For example, one observation is that every item in the Large group must be assigned its own individual bin. The main focus of MFFD is to efficiently place the contents of the Medium and Small groups into these Large-item bins (sorted largest to smallest).

First, a forward pass is done, placing the largest Medium item that will fit into each bin; then a backwards pass is done for any skipped bins, placing the largest and smallest Small item that will fit into them. Finally, each bin is packed as much as possible with the largest remaining items. Once all Large bins have been maximally filled, any remaining items are binned using ordinary FFD. This algorithm is longer and more complicated that simple FFD, yet it still runs in $O(n^2)$ time, keeping it way ahead of exact algorithms such as ILP. For clarity, that's $O(n \log n)$ for sorting/grouping, $O(|\text{Large}| \cdot |\text{Med}|)$ for the first loop, $O(|\text{Large}| \cdot |\text{Small}|)$ for the second loop, and $O(|\text{Large}| \cdot (|\text{Medium}| + |\text{Small}| + |\text{Tiny}|))$ for the third loop, all well-within $O(n^2)$. Additionally, MFFD boasts the *best* approximation factor of all approximate BP algorithms at 1.183:

$$\text{MFFD}(I) \leq \frac{71}{60}\text{OPT}(I) + 1$$

FFD Implementation

As the utility of **Approximate** algorithms strongly rest upon their efficiency, here we specify the design choices made in the FFD and MFFD C++ algorithm implementations for the sake of optimisation.

The FFD implementation is quite compact. It takes a `vector` of integers (assumed positive) and sorts it (using `std::greater` to automatically sort in descending order). As this algorithm specification simply counts the number of used bins, rather than the actual item assignment, the bins are represented as a `list` of integers, representing the remaining capacity for unplaced items. The algorithm then iterates on the numbers in descending order, and uses an `iterator` to iterate the bin `list`, determining if the remaining capacity permits placement. If all bins cannot fit the given item, a new bin is pushed onto the `list` with the corresponding capacity. All that's left to do afterwards is to count the number of bins opened during the algorithm to retrieve the closely-approximated answer.

The MFFD implementation is slightly more involved. Instead of sorting the item `vector` in-place, the algorithm automatically sorts them by grouping them into `multisets` of each item size class. The Large-class items are automatically assigned a bin and then the three loops of the algorithm as defined above take place. These loops run entirely using `iterators`, and are thus quite efficient and can parse very large systems in milliseconds. The rest of MFFD runs as expected, as well as a final call to FFD for any remaining items.

Experimental Methodology

Test Instances

Rather than generating our own instances, we used the BP benchmark library BPPLIB (Delorme et al., 2018), which collects problem instances used in most of the Bin Packing literature all together with solutions provided. This gives us two things which a self-generated set wouldn't: A wide rang of instance families with known, documented structures and a trusted reference value of OPT(I) against which both algorithms can be checked. All 6,345 BP instances (ignoring alternate-format duplicates) were downloaded from the library's git repository and was included to our own repository under `Data/bpplib/`, as one folder per set. Table 1 summarises the sets.

Sets	Instances	n	C	Structure
Falkenauer_U	80	120 - 1000	150	Uniformly-distributed sizes
Falkenauer_T	80	60 - 501	1000	Triplets, every optimal bin holds exactly 3 items
Scholl_1	720	50 - 500	100 - 150	Uniform sizes, small capacity
Scholl_2	480	50 - 500	1000	Few items per bin
Scholl_3	10	200	100,000	Very large capacity
Wascher	17	57 - 239	10,000	Hard literature instances
Schwerin_1, _2	200	100, 120	1000	Narrow size range
Hard28	28	160 - 200	1000	Known difficult instances
Randomly_Generated	3840	50 - 1000	50 - 1000	Controlled grid of n, C and size range
AI, ANI	500	201 - 1003	up to 80,000	Designed to be hard to prove optimal
GI	240	1176 - 5486	up to 1.5×10^6	Very large instances
SSTriples_216, _405, _648	150	216 - 648	10,000 - 20,000	Triplet structure, large-scale
Total	6345	-	-	-

Table 1: BPPLIB instance sets used in experiment

Concerning the ILP algorithm, the number of items n controls the size of the ILP, as the model has $U + nU$ variables. The capacity C and the item size range control how many items fit in a bin and therefore how easily small items can fill gaps, which is what FFD relies on.

The Triple sets (Flakenauer_T, SSTriples) reproduces the 3-partition structure behind the problem's strong NP-hardness, where every bin must be filled exactly and a single misplaced item costs an extra bin. The AI and ANI sets were built so that an optimal packing is hard to prove (Delorme et al., 2016, p.10), isolating the weakness of ILP's lower bound. Finally, the Randomly_Generated set lets us vary one parameter at a time as it contains 10 instances for every combination of $n \in \{50, 100, 200, 300, 400, 500, 750, 1000\}$, and $C \in \{50, 75, 100, 120, 125, 150, 200, 300, 400, 500, 750, 1000\}$, with the maximum item sizes as $\{0.7C, 0.8C\}$, with uniformly-random sizes (Delorme et al., 2016, p.10).

The Library's solutions file are stored in our repository in `Data/bpplib_solutions.csv`, recording each instance's best known lower bound, best known upper bound, and whether the two are equal (whether the instance is Solved). For Solved instances, the upper bound is OPT(I). This is the case for all but 58 of the 6,345 instances.

Benchmark rig

Both algorithms are run in the same C++ program so that they're timed properly, in the same process, on the same machine, and over identical inputs. It's invoked as `./binpack <time limit> <set> [set ...]` and works in three stages.

Loading: The loader reads every file in the input problem set in sorted order, so every run processes instances in the same sequence. Each file is validated as it is read, where the item count and capacity must be positive and the declared number of sizes must be present, requiring every item size to lie between 1 and C. Any malformed file stops the run with an error. Each instance is then matched with its known optimum from the solutions file. All instances are loaded before any timing begins, so file input handling is never measured as part of the benchmarking process.

Running: For each instance, the rig runs FFD and MFFD first. This allows the ILP to take the FFD's bin count as an approximate input as an optimised starting-point for U. Each call of the three program implementations is timed separately using `std::chrono::steady_clock`, a monotonic clock independent of system time, and is recorded in milliseconds. FFD's time includes sorting the items, though it is noted that the vast majority of problem instances are already given pre-sorted. The ILP's time includes both building the model and solving it (via HiGHS), since both are part of the methods cost, though it does not include the FFD run used to get U. Additionally, the ILP algorithm takes given a time-out limit to prevent it from taking too long (10 seconds, in the main experiment). If HiGHS has not found any feasible packing by then, the ILP reports -1. This results in the ILP potentially providing a non-optimal output value when its time-out limit elapses, providing the best feasible value it has found in the period it was given.

Recording: Each problem instance produces one row in the output file `results/<set>.csv`, containing the instance name, n, C, known optimum, as well as each algorithm's output bin count (and time limit flag for ILP's case). Rows are flushed to disk as soon as they are written, so even if a long run is interrupted, it saves its completed results. The command `make run` runs every set.

Experimental Setup

All experiments were run on a MacBook Pro with an Apple M3 Pro processor and 18 GB of memory, compiled via clang++ using `-std=c++17 -O2` and linked against HiGHS v1.15.1 installed via Homebrew. Additionally, HiGHS was restricted to a single thread so that it performs on equal terms with the FFD algorithm.

Experimental Considerations

Both algorithms receive exactly the same problem instances, which are read once into the memory before any timing begins. Several aspects of the comparison are nevertheless uneven and we state them here so that the results can be interpreted accurately. First, the implementation efforts are not directly comparable; FFD is Elan's own code, whereas ILP delegated its search to HiGHS, a solver developed over many years. Nevertheless, we consider this appropriate, as our technical question is how the two approaches compare in practice, though it also means that ILP represents a well-engineered, exact method rather than a hand-written one. Second, the ILP is not entirely independent of FFD, since FFD's bin count output sets the number of bin slots U for ILP. Third, to hasten the experimental procedure, `make run` executes several problem sets in parallel on separate cores. As the processes share memory bandwidth and the M3 Pro adjusts its clock speed dependent on processing load, absolute time for a given problem instance may vary somewhat between runs. Finally, regarding a consideration we only noticed after the execution of the experiment, FFD often finishes in just a few microseconds, very close to the resolution of the clock, so these individual FFD times should be read as orders of magnitude rather than precise values.

Results and Analysis

Once the experimental methodology was fully completed, we wrote a Python script to read through the result tables and produce a comprehensive set of graphs, visualising the accuracy, error, and runtimes for each of the three algorithms across the entire base of problem sets. The full set of charts and graphics for all datasets is available in our repository under `results_analysis/graphs`.

Algorithm Accuracy

First, we compared each algorithm's performance of their output bin value to the known optimal. Figures 1 & 2 demonstrate the typical behaviour of each of the three algorithms. The Wascher dataset contains just over a dozen problem instances with very tricky-to-optimise solutions, and yet, none of the algorithms ever overestimate the optimum by more than 1; in fact, all algorithms produced the same output on this problem set (ILP was able to determine the FFD's best bin assignment but could not find a better assignment before timing out). This close approximation structure could also be seen in the GI problem set for the FFD and MFFD algorithms, with at most 51 and 48 extra bins, respectively. However, the large problem size completely overwhelms the ILP algorithm, timing out before it could even find a single feasible, suboptimal solution.

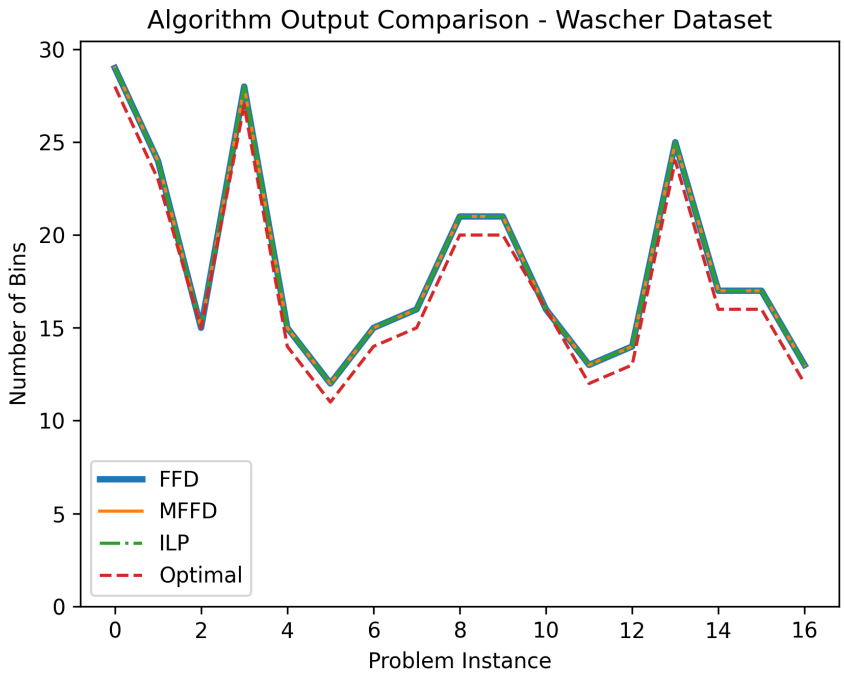


Figure 1: Bin Algorithm Output: Wascher Dataset

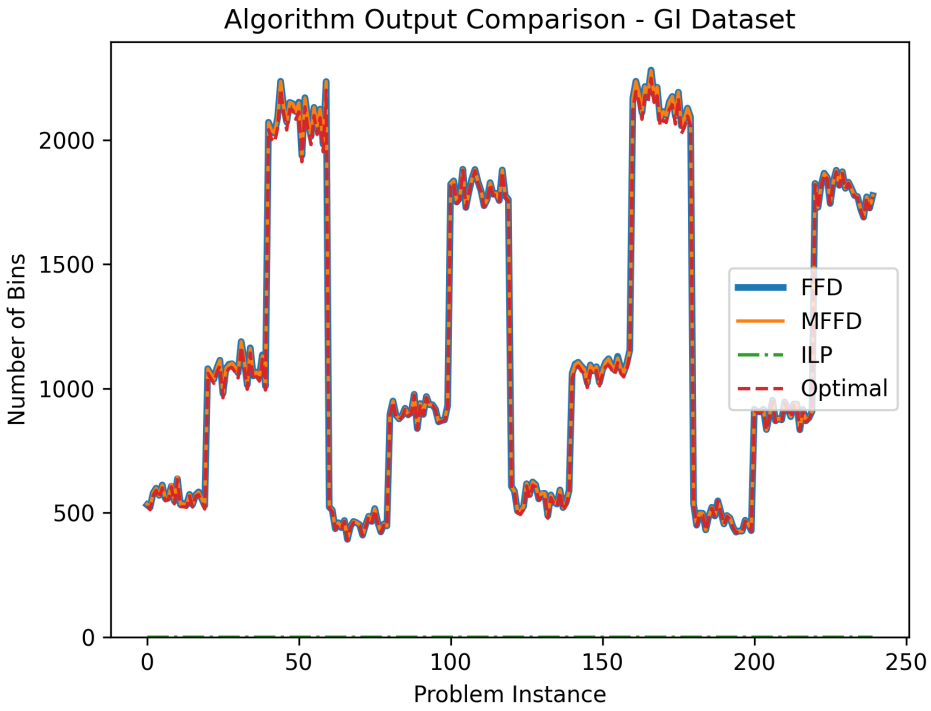


Figure 2: Bin Algorithm Output: GI Dataset

In contrast, Figure 3 visualises the algorithms' performance on the heavily-constrained Falkenauer_T dataset, demonstrating the largest discrepancy our experiment has produced between the algorithms and the optimal value. It exaggerates the general pattern present across all 16 datasets; the ILP algorithm is often the most accurate (to no-one's surprise) but does not often have enough time to reach the optimal value. Meanwhile, the FFD-based algorithms are not too far behind ILP, and remain very close to each other. The problem set that best amplifies ILP's non-optimal performance is the Scholl_3 dataset as seen in Figure 4. In this dataset, all three algorithms are generally stuck 4 bins away from the optimal, with ILP only getting the closest at 2 bins away.

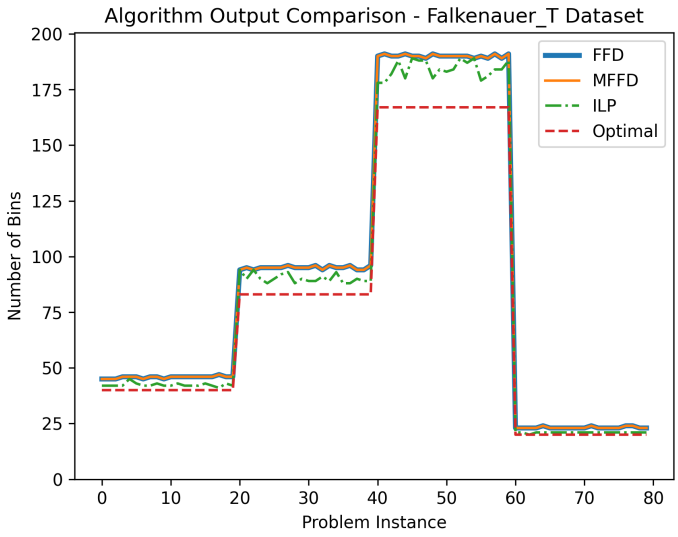


Figure 3: Bin Algorithm Output: Falkenauer_T Dataset

Overall, ILP performed the best on average on small-to-medium-sized problem instances, though was beat out by FFD when it failed to finish on instances with large bins and numerous items. Analysing all 6345 problems and counting how many each algorithm achieves the optimal, we get 2647 for FFD, 797 for MFFD, and 2519 ILP. While ILP is generally more accurate on problems (primarily due to its reliance on FFD's solution), it is unable to reach optimality more often than FFD, particularly in the Randomly_Generated dataset (FFD: 1601, MFFD: 419, ILP: 1502), likely due to timing out before any feasible solution can be attained and returning a non-answer.

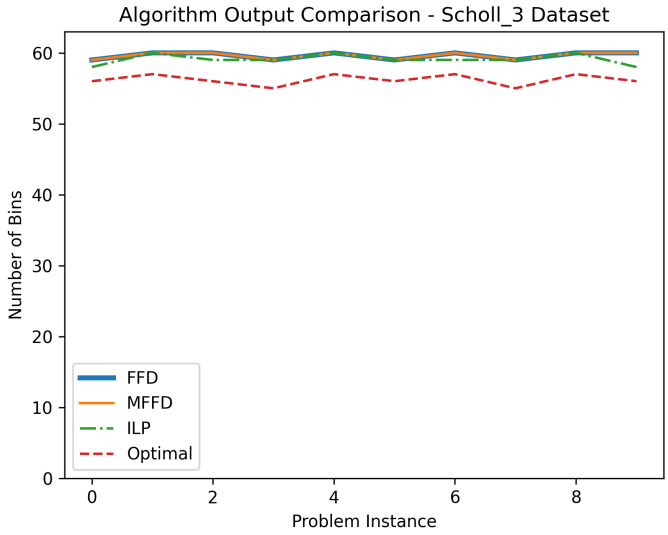


Figure 4: Bin Algorithm Output: Scholl_3 Dataset

Algorithm Error

We decided on two main ways to analyse the error between each algorithm's output number of bins vs the known optimal. First we compare the values additively, counting how many bins each went over the limit, which works best to describe smaller problem instances in finer detail. Afterwards, we compare the values multiplicatively, giving a better sense of the overall approximation factor performance across the board.

In Figure 5, the additive errors across all datasets has been collated and averaged, depicting their performance against the optimal. Note that for this graph in particular, we have filtered out all ILP runs except those that found a feasible output. A few trends can be thus identified; FFD's average performance is always better than or equal to that of MFFD's, and is always worse or equal to that of ILP's. The only dataset with perfect optimality from an algorithm was FFD on ANI, solving all 250 problems exactly. ILP timed out on 68% of this dataset, but of the 81 problems it solved, it too reached optimality on every one. Interestingly, despite handling problems with several hundred bins, every algorithm performed with an average just above 12 over the optimum, even in the most difficult problems.

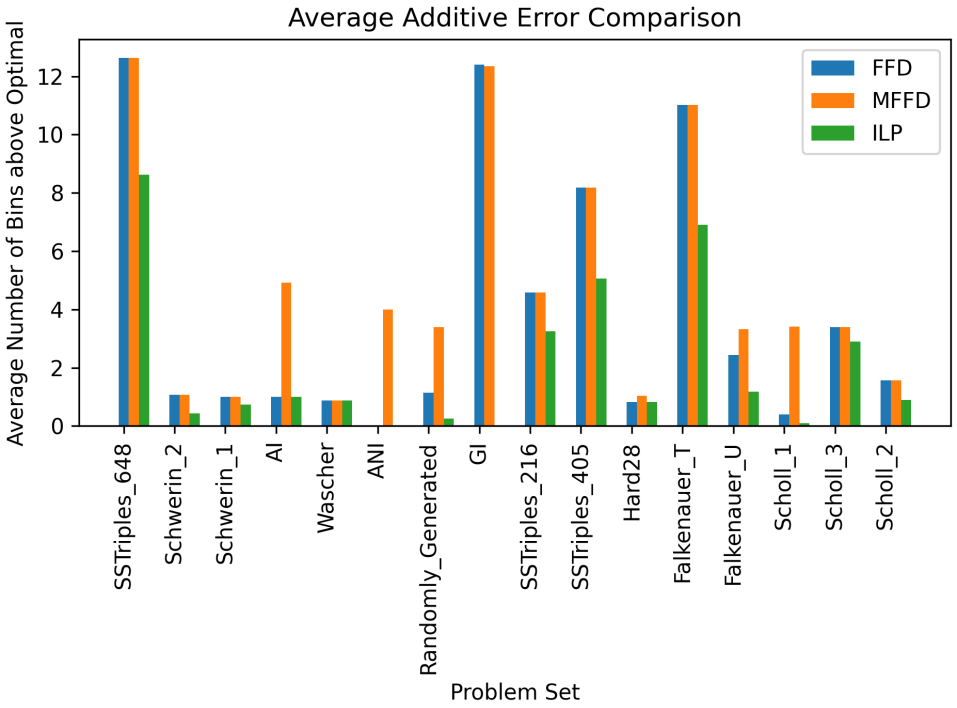


Figure 5: Average Additive Error Summary

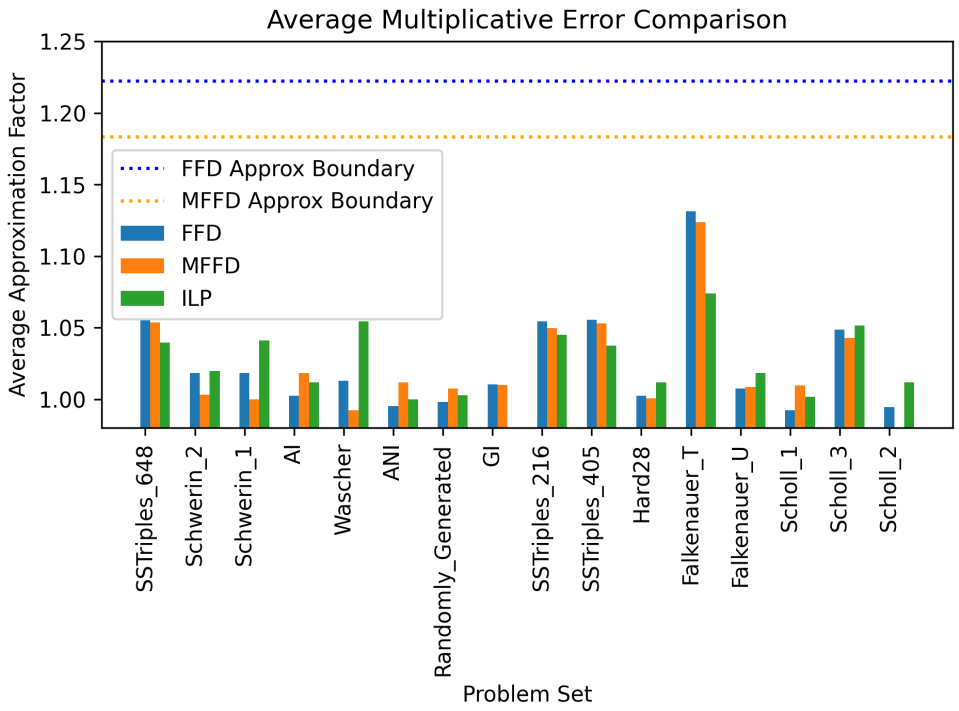


Figure 6: Average Approximation Factor Summary

As for multiplicative error, a corresponding chart is depicted in Figure 6. Since the FFD-type algorithms have known, proven asymptotic approximation factor bounds, they have been included in the figure, showing that neither approximate algorithm got particularly close to their boundary. Looking closer at each problem set individually, we can substantiate that both FFD and MFFD never exceed their boundary. The closest that the algorithms got were when the problem instances were constrained but small, such as in the Falkenauer_T dataset as seen in Figure 7. However, all algorithms' performances tend to remain relatively close to 1, especially for larger problem instances, as seen in Figure 8.

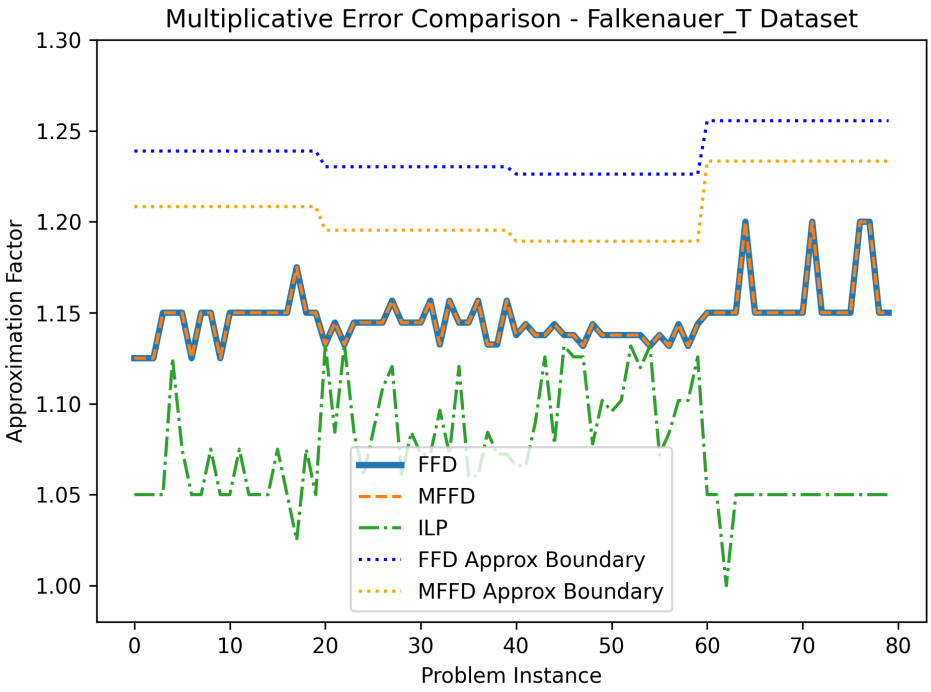


Figure 7: Algorithm Approximation Factor: Falkenauer_T Dataset

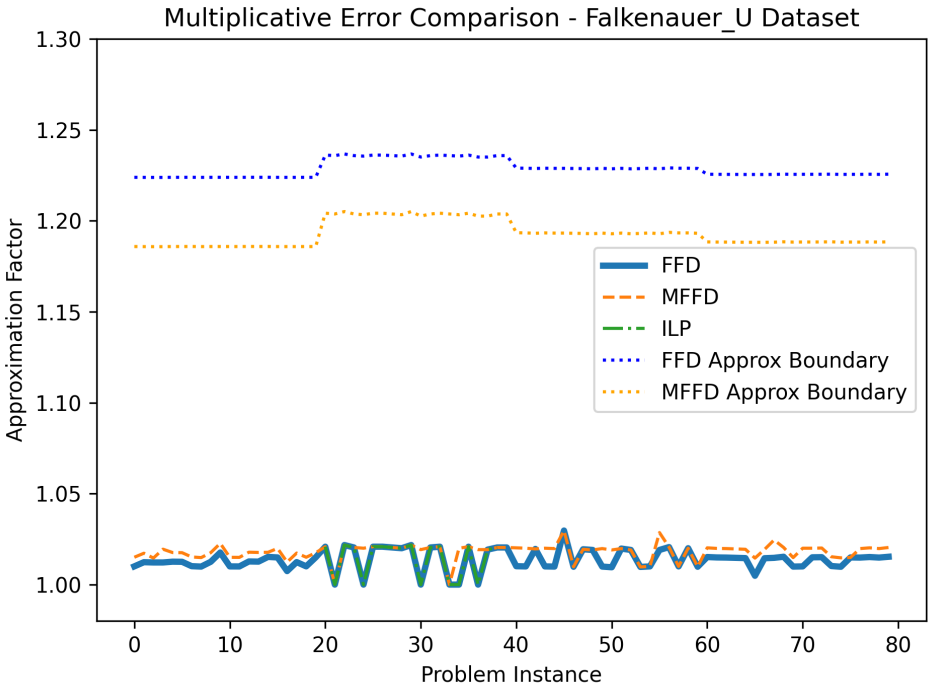


Figure 8: Algorithm Approximation Factor: Falkenauer_U Dataset

Algorithm Runtime

The algorithm runtime performance had the starkest contrast in performance between the two methods. Figure 9 summarises the average runtime across all problem sets, and the difference is rather immediate, requiring a logarithmic scale to even represent both methods simultaneously. Both FFD algorithms perform very quickly, very consistently, taking less than a millisecond on average for all algorithms, except for the GI problem set, containing the largest problem instances in the entire library. Meanwhile, ILP is persistently stuck at the 10 second mark across all problem sets, fitting expectations since it was unable to consistently achieve the exact optimum before timing out.

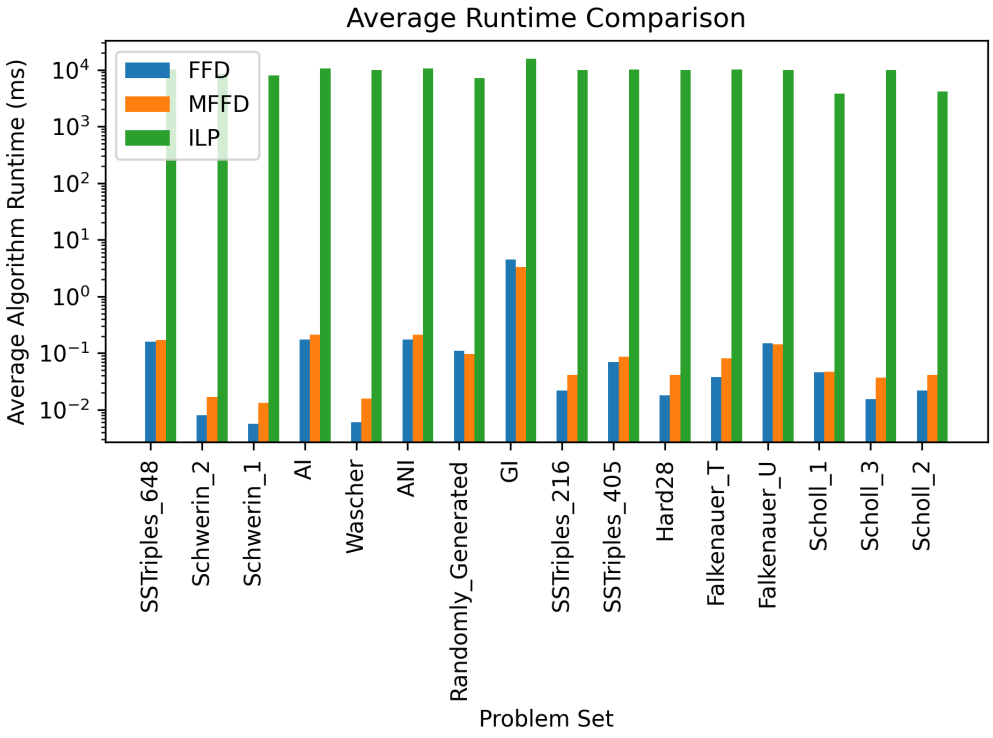


Figure 9: Average Runtime Summary

The biggest observable disparity in runtime is visible in the Wascher dataset in Figure 10, where ILP always times out but the FFD algorithms never take longer than 0.04ms. Comparatively, the Scholl_1 and Scholl_2 datasets provide the clearest instances of the ILP algorithm terminating before the time-out (Figures 11 & 12), with the fastest instances finishing in about 5ms.

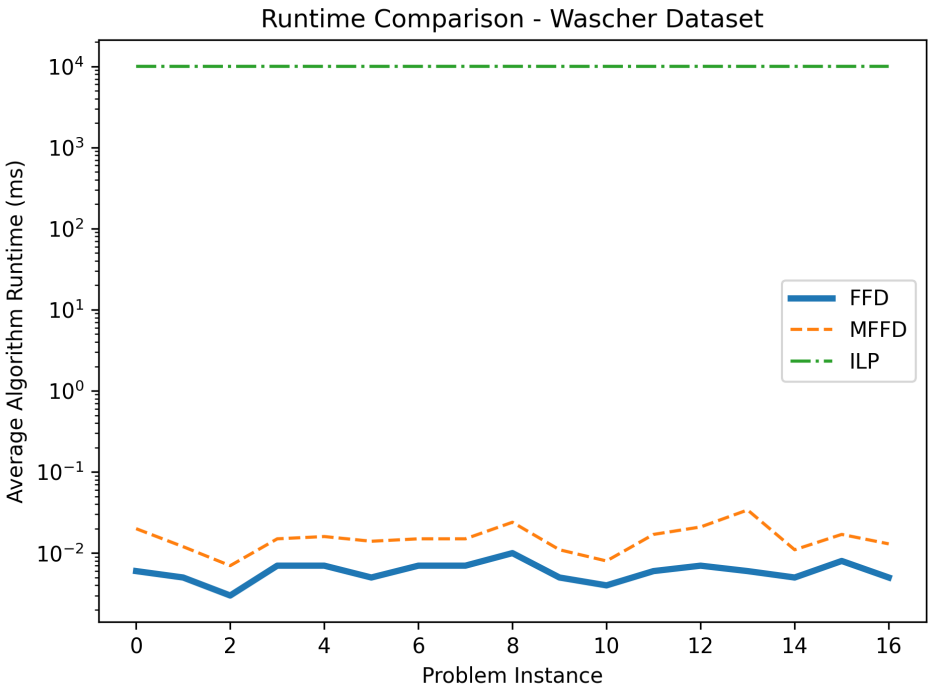


Figure 10: Algorithm Runtime: Wascher Dataset

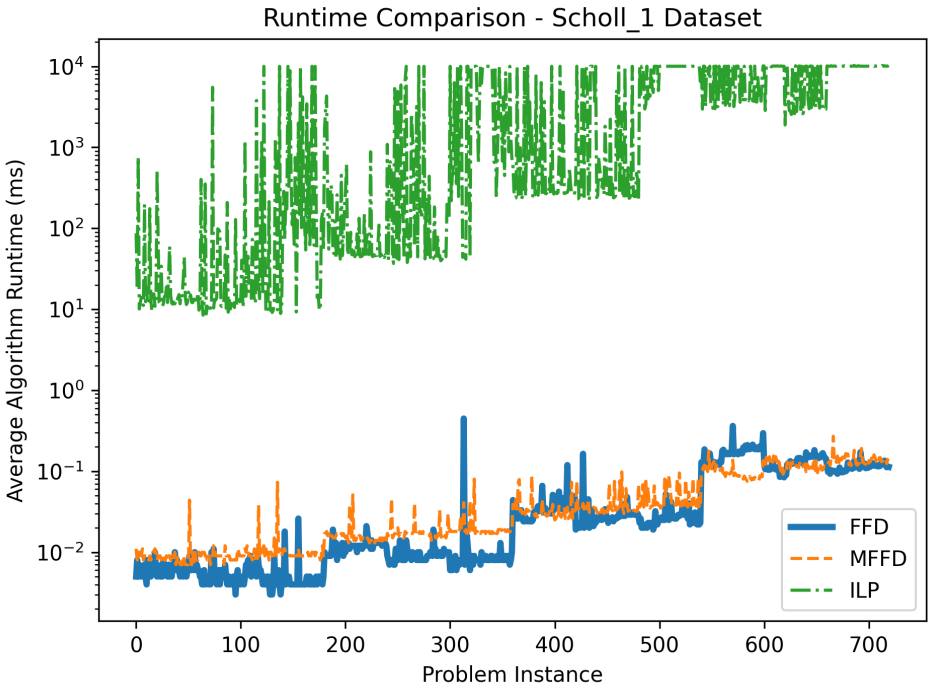


Figure 11: Algorithm Runtime: Scholl_1 Dataset

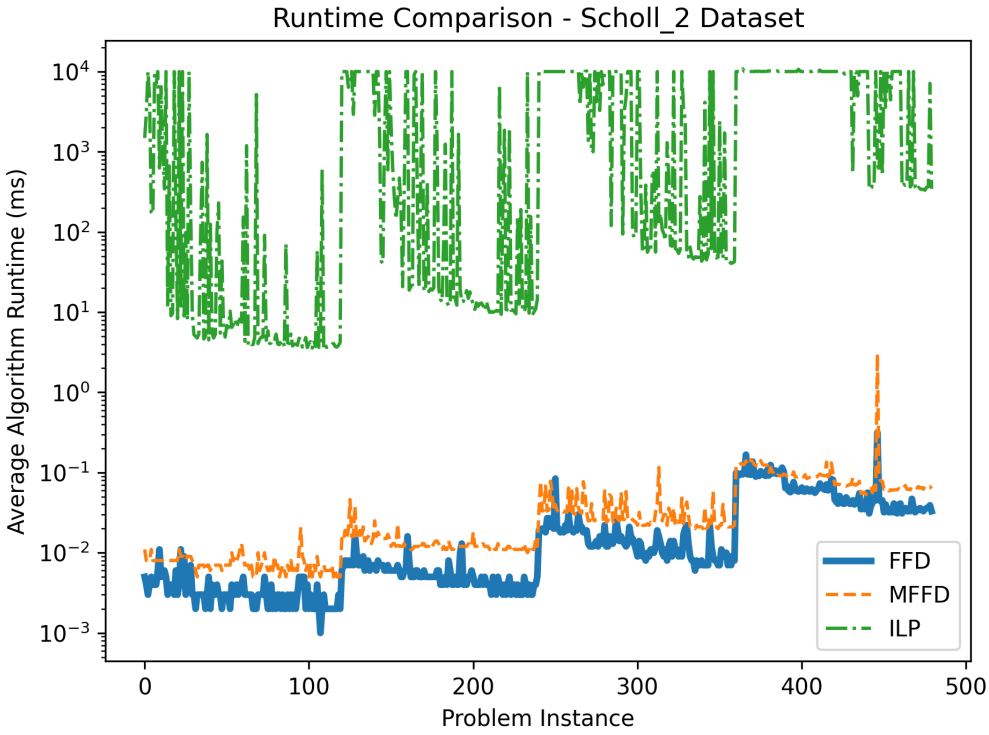


Figure 12: Algorithm Runtime: Scholl_2 Dataset

Overall, we consider the FFD algorithm to have had the overall best performance. It was consistently and reliably quick, and was always well-within its expected $\frac{11}{9}$ approximation factor, only off by a couple bins overall, not mentioning the simplicity in the actual algorithm procedure. While ILP had some strong potential as a rigorous exact algorithm, it's runtime ballooned way too quickly to be practical for the problem instances in our test library, and thus never got to achieve its optimum-finding capabilities. MFFD's performance is a bit more quizzical; it constantly performed slightly worse than FFD, despite being proven to have a tighter approximation factor. It seems that none of the problem instances in our library were exceptionally tight on the $\frac{71}{60}$ factor, so MFFD never had an opportunity to surpass FFD's simpler bound.

Surprises

How early ILP hits the wall: We had expected before running the experiment for FFD to be significantly faster than ILP, but we did not expect the gap to be this large, or for ILP to struggle on small problem instances as much as it did. FFD's median time per instance was about 0.05 ms, while ILP's median on the instances it was able to solve was about 214 ms, i.e. more than 4000 times slower, even ignoring the majority of executions that timed out. More surprising was how quickly ILP stopped finishing as the problem sizes grew. With only $n \leq 50$ items, 4% of instances already hit the 10 sec limit. This rose to 32% for $n \leq 100$, 61 % for $n \leq 200$, 83% for $n \leq 500$ and 96% for $n \leq 1000$, and every instance above 1000 items invariably times out. The model has only $U + nU$ variables, so it's not even abnormally big in absolute terms. The blow-up comes from the branch-and-cut search tree, not from the size of the model.

ILP failed on instances FFD solved perfectly: The cleanest example of this is the ANI set, where FFD found the optimal packing for all 250 instances, and ILP hit time limit on all 250. Across the whole library, this outpacing happened on 832 different problem instances. We suspect that this may be a consequence of the problem instances' structure, since in every ANI instance, the optimum is exactly one bin above the lower bound $\sum_i y_i/C$. ILP actually found the optimal packing on 81 of these problems, but couldn't prove it within 10 seconds. Closing that 1-bin gap is exactly where the model's weak LP relaxation lets it down. In practice, most of ILP's time goes into certifying the answer, not finding it.

MFFD performed worse than FFD: On paper, MFFD should beat FFD, its guaranteed approximation factor ($\frac{71}{60}$) is tighter than FFD's ($\frac{11}{9}$), and it never broke that guarantee on any instance. Though in practice, it used more bins than FFD on 3,737 instances and did better on only 42. One explanation is that a guarantee only describes the worst case scenario, rather than typical inputs and MFFD's fixed-size class rules may leave more gaps than FFD's simple "largest item first" approach. However, despite testing the algorithm to our satisfaction, we cannot yet rule out with certainty that there may be a bug in our implementation responsible for this subpar performance.

Next Steps

More precise timings: FFD often finished in a few microseconds, closer to the resolution of the clock. To gain more precision in our measurements, we could run FFD and MFFD hundreds to thousands of times per instance and average the results, cancelling out the majority of instrumental noise. for ILP, we could re-run the HiGHS solver with `make run JOBS = 1` instead of several sets in parallel, so processes don't compete for memory bandwidth and M3 Pro's clock speed stays steady, though at the cost of making the whole experiment take longer to execute.

Warm-starting ILP w/ FFD's packing: Depending on what criteria we would want to investigate in ILP's performance, we have two different options on how to prepare the initial

LP solution. On one hand, if we wanted to get an exact, optimal solution in as little time as possible, we could run a variation of the FFD algorithm that produces the packing assignment, rather than just the number of bins. Currently, FFD only sets the number of ILP bin slots U. Passing an actual packing to HiGHS as an initial solution would mean that the ILP always has an answer at least as good as FFD's and would dissociate the cost of finding a packing from the cost of proving it optimal.

Cold-starting ILP: In complete contrast, if we wanted to judge ILP's performance as a standalone algorithm, we could skip the pre-processing of finding an approximate FFD value and have the ILP solver work out the entire system from $U = n$. This would be the clearest benchmarking of the ILP algorithm, however we would expect it to have an incredibly long runtime on relatively small problem instances (which is why we did not choose to do so in this methodology). This version of the experiment would best be done with an external processing resource with plenty of compute time.

A stronger formulation: The ILP model's weak LP bound is what made instances like ANI so hard. (Delorme et al., 2016) demonstrated that arc-flow and set partitioning (column generation) ILP models have much tighter bounds. Comparing one of them against our HiGHS model implementation could help determine how much of the ILP's poor performance comes from the problem formulation, rather than a blanket inefficiency in any exact algorithm method.

Tighter FFD bound problem instances: As we never observed either of the FFD-type algorithms reach either of their respective approximation limits, we never got to completely analyse the quality of their approximations. For example, MFFD rarely performed better than FFD did, despite having the better theoretical limit. An improvement to our problem library would be to derive another problem set designed to cause the FFD algorithm to hit its $\frac{11}{9}$ limit (Dósa, 2007). The Falkenauer_T problem set got close, but not close enough. These proven-tight-bound instances would reliably demonstrate whether the MFFD algorithm is able to find solutions within its $\frac{71}{60}$ bound, or if it fails.

References

Delorme, M., Iori, M., & Martello, S. (2016). Bin packing and cutting stock problems: Mathematical models and exact algorithms. *European Journal of Operational Research*, *255*(1), 1–20. https://doi.org/10.1016/j.ejor.2016.04.030

Delorme, M., Iori, M., & Martello, S. (2018). BPPLIB: A library for bin packing and cutting stock problems. *Optimization Letters*, *12*(2), 235–250. https://doi.org/10.1007/s11590-017-1192-z

Dósa, G. (2007). The tight bound of first fit decreasing bin-packing algorithm is ffd(i) ¡= 11/9opt(i) + 6/9. In B. Chen, M. Paterson, & G. Zhang (Eds.), *Combinatorics, algorithms, probabilistic and experimental methodologies* (pp. 1–11). Springer Berlin Heidelberg.

Dósa, G., & Sgall, J. (2013). First Fit bin packing: A tight analysis. In N. Portier & T. Wilke (Eds.), *30th international symposium on theoretical aspects of computer science (stacs 2013)* (pp. 538–549, Vol. 20). Schloss Dagstuhl – Leibniz-Zentrum für Informatik. https://doi.org/10.4230/LIPIcs.STACS.2013.538
Keywords: Approximation algorithms, online algorithms, bin packing, First Fit.

Garey, M. R., & Johnson, D. S. (1975). Complexity results for multiprocessor scheduling under resource constraints. *SIAM Journal on Computing*, *4*(4), 397–411. https://doi.org/10.1137/0204035

Garey, M. R., & Johnson, D. S. (1979). *Computers and intractability: A guide to the theory of NP-completeness*. W. H. Freeman.

HiGHS Development Team. (2026). *HiGHS documentation* (1.15.1). Retrieved October 29, 2026, from https://ergo-code.github.io/HiGHS/stable/

Huangfu, Q., & Hall, J. A. J. (2018). Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, *10*(1), 119–142. https://doi.org/10.1007/s12532-017-0130-5

Johnson, D. S., & Garey, M. R. (1985). A 7160 theorem for bin packing. *Journal of Complexity*, *1*(1), 65–106. https://doi.org/https://doi.org/10.1016/0885-064X(85)90022-6

Silvano, M. (1990). *Knapsack problems : Algorithms and computer implementations*. J. Wiley & Sons. https://web.archive.org/web/20060508092838/http://www.or.deis.unibo.it/kp/Chapter8.pdf

Individual Contributions

Elan Blaustein:

- The Bin Packing Problem
- First-Fit Decreasing
- Results and Analysis
- Next Steps
- Editing

Swas Dwivedi:

- The Bin Packing Problem
- Integer Linear Programming
- Experimental Methodology
- Surprises
- Next Steps

AI Use Declaration

Swas Dwivedi: I've made use of AI through this assignment, largely for two parts of this project: early research and writing the benchmarking infrastructure.

Initially, I used Clause chat to research more about the BP problem and how it can be solved via ILP, as in how it can be written as an Integer Linear Program. The assignment's description lists "ILP formulation with a solver" as an option for the exact algorithm, which I further discussed with Claude and settled on HiGHS, an open source ILP solver with a C++ API that we could directly call onto. Additionally, it was also very useful for understanding the HiGHS documentation.

However, it did miss the context of how the model is to be used. Its first formulation was the standard assignment model with a binary y_j for each of the n possible bins, a binary x_{ij} for each item per bin, and a capacity constraint of $\sum s_i.x_{ij} \leq C.y_j$ for each bin. This is theoretically correct, but what Claude missed is that the model is to be run over BPPLIB with hundreds of instances under a 10 sec time limit. Because every relabelling of the model gives an equally good solution, the plain model literally wastes most of its search on identical packing and I therefore resorted to capping the number of bins using FFD's result. Additionally, I sorted items by decreasing size, so that "item i may go in bins $\{1...i\}$", which prunes the search and made `mip_abs_gap` just below 1, as the objective is a whole number such that any gap below 1 already proves optimality.

I also used Claude code to generate the benchmarking rig (`benchmark.cpp`, `problems.cpp`, the test harness, and parallel `make run`) as noted in commit `42b8752`. I read and ran the generated code, which Elan reviewed independently.

From my perspective, AI is useful as long as it is either a reference or a way to offload laborious work, not a way to offload cognitive work. In our group, it helped most by letting us both focus on our own algorithms. Claude code produced a working, if slightly flawed version of the benchmarking rig which otherwise would have had been added onto someones workload. Once Elan's FFD had a stable signature, adding it onto the shared rig,took minutes and we could run all 6,345 instances through identical inputs in one process. The cost was that neither of us had written the rig by ourselves, so its flaws were easy to miss.

Elan Blaustein: In this assignment, like others that I have previously completed, I have actively avoided the direct usage of AI tools for the process of learning, programming, and documenting my assigned algorithm. I personally find more utility from the endeavour of working through the problem, as I believe it better supplements my learning process.

However, I am cognisant of Swas' usage of AI tools for the programming and development of the ILP algorithm, as well as the underlying structure of the benchmarking rig, and that the results of our investigation rely on the quality of these AI tools. I have scrutinised the code developed by AI and verified its output, making adjustments where necessary in areas the tools overlooked. For example, I changed the problem testing criteria to mark the FFD algorithm's output as a 'pass' as long as it remains within the expected approximation factor, rather than failing if the output did not meet the exact optimal result.

From my perspective, the most utility that our group derived from the use of AI tools was primarily the accelerated development of code (which I vicariously benefited from). Since the ILP implementation used an external LP software package, being aided by an AI system to automatically configure all the HiGHS settings streamlined our process greatly. However, the use of AI here always risks the threat of swallowing the project whole; first it writes the boilerplate, but then it actively offers to build the rule generation, writing out the human-in-the-loop. As with any situation involving the use of AI, the key to mindful operation is to identify the boundary where AI's influence should terminate and abide by it.